

Instituto Tecnológico de Buenos Aires (ITBA)



22.07 - Programacion I

Resumen de C

Ignacio Sammartino

Índice

Names in C	3
Function that exits a program - exit()	3
printf and scanf	3
Scaler types	4
Primitivos - Enteros	4
Primitivos - Punto Flotante	5
Casteo (Implicitos)	5
Notación Científica	5
Define	5
Enumeradores	5
Logica Booleana	5
TypeDef's	6
Multidimensional arrays	6
Pointers	7
Punteros y arreglos	7
Aritmetica de punteros	8
Equivalencias	8
Strings	8
True vs False	9
Switch	9
Logical operators	9
Decrement and increment operators	9
Comparing floating point values	9
Bit operators	9
sizeof operator	9
Structures	9
Structure operators	9
Unions	9
Dynamic memory allocation on the Heap	9
Types of includes	9
Keyword static	9
Same ways of defining strings	9
Functions for manipulating strings and general functions	9
Functions for manipulating files	9
Function pointers	9
C99 - stdint.h - Primitive fixed size types	9
Preprocessor Macros	9
Technique for defining more than one statement in a preprocessor Macro.	9
C for Embedded Systems	9
Keyword const and volatile	10
Sizes for 32 bit microController	10

Two ways of making a Menu with strings - char * -
and send it to UART. 10

Names in C

En el lenguaje C, los identificadores (nombres de variables, funciones, etc.) deben seguir reglas específicas:

- Son **case sensitive** (distinguen entre mayúsculas y minúsculas).
- Deben comenzar con una **letra** o un **guion bajo** («_»).
- Solo pueden contener **guiones bajos, letras o números**.

```
int variableValida;  
int _contador;  
int valor2;  
// int 2valor; // Esto causaría un error de compilació
```

Function that exits a program - exit()

exit() La función exit() permite terminar la ejecución de un programa de forma inmediata desde cualquier punto. Se encuentra definida en la librería <stdlib.h>. Comúnmente se utiliza el valor 0 para una terminación exitosa y valores distintos para indicar errores.

```
#include <stdlib.h>  
  
int main(void){  
    // Exit successfully. The program terminates at this point.  
    exit(0);  
  
    // Exit with error.  
    exit(-1);  
}
```

printf and scanf

Estas son las funciones fundamentales para la entrada y salida de datos en consola, incluidas en la librería <stdio.h>.

- printf(): Se utiliza para imprimir texto y variables con formato.
- scanf(): Se utiliza para leer datos del usuario desde el teclado.

```
#include <stdio.h>  
  
int main() {  
    int edad;  
    printf("Ingrese su edad: ");  
    scanf("%d", &edad); // El símbolo & es necesario para obtener la dirección  
    printf("Su edad es: %d años\n", edad);  
    return 0;  
}
```

De querer escanear otro tipo de input para `scanf()`, tengo las siguientes opciones:

- `%d` : integer
- `%f` : floating point
- `%c` : char
- `%s` : NULL or “0” terminated string
- `%x` : Hexadecimal number
- `%o` : Octal number

Scaler types

Los tipos escalares en C representan valores individuales y se dividen en varias categorías según las fuentes:

- **Tipos primitivos:** Incluyen los tipos básicos como `int`, `char`, `float` y `double`.
- **Notación científica:** C permite representar números reales usando notación científica (ej. `1.2e3`).
- **C99 - `bool`:** A partir del estándar C99, se incluye el tipo booleano mediante la librería `<stdbool.h>`.
- **Enums:** Permiten definir tipos de enumeración para manejar constantes relacionadas.

```
#include <stdio.h>
#include <stdbool.h>

enum dia_semana {LUNES, MARTES, MIERCOLES};

int main() {
    int entero = 10;
    float pi = 3.14159;
    double cientifica = 1.2e-3;
    bool esVerdad = true;
    enum dia_semana hoy = LUNES;

    return 0;
}
```

Primitivos - Enteros

type	size (bits/bytes)	Range
unsigned char	8 / 1	0 to 255
char	8 / 1	-128 to 127
unsigned int	16 / 2	0 to 63535
short int	16 / 2	-32768 to 32767
int	16 / 2	-32768 to 32768
unsigned long	32 / 4	0 to 4294967295
long	32 / 4	-2147483648 to 2147483647

Primitivos - Punto Flotante

type	size (bits/bytes)	Exponente	Mantissa
float	32 / 4	8	23
double	64 / 8	11	52

Casteo (Implicitos)

Esto ocurre cuando se le asigna un valor a una variable, y el tipo de la constante no coincide con el de la variable. En dicho caso, el tipo de la constante es casteado (promovido truncado) al tipo de la variable

float	x = 3;	//almacena el 3.0	C
int	i = 8.452	//almacena el número 8	
char	a = 200	//(número negativo)	
unsigned char	b = -12	//(número positivo)	
int	c = -12.89	//(almacena el -12)	
int	e = '5'	//(almacena el ASCII del 5)	

Notación Científica

// Notacion Cientifica	C
float num_2 = 5e-5;	
double num_3 = 3.7e12;	

Define

La directiva #define que nos permite asignarle a un símbolo un valor

# define NUM_A 5.5	/* Double constant */	C
# define NUM_B 5.5f	/* Float constant */	
# define NUM_C 5.5e6L	/* Long double constant */	
# define ADDRESS 0x20004000UL	/* Unsigned Long constant */	

Enumeradores

Por default, LUNES arranca en 0

enum dias { LUNES, MARTES, MIÉRCOLES, JUEVES, VIERNES, SÁBADO, DOMINGO }	C
--	---

De queres forzar a que arranque desde otro numero.

enum dias { LUNES = 1, MARTES, MIÉRCOLES, JUEVES, VIERNES, SÁBADO, DOMINGO }	C
--	---

Logica Booleana

#include <stdbool.h>	C
bool flag = false;	
flag = true;	
if (flag){ print("Flag is true!\n");}	

TypeDef's

La palabra clave typedef se utiliza para crear un alias o un nuevo nombre para un tipo de dato ya existente, lo que mejora la legibilidad del código.

```
typedef unsigned long ulong;
typedef int entero;

int main() {
    ulong numeroGrande = 1000000;
    entero edad = 20;
    return 0;
}
```

Multidimensional arrays

C permite la creación de arreglos de múltiples dimensiones (matrices). Al pasarlos a funciones, es importante recordar que se deben especificar las dimensiones **excepto la primera**.

```
void imprimirMatriz(int matriz[2][1]) {
    for(int i = 0; i < 2; i++) {
        for(int j = 0; j < 3; j++) {
            printf("%d ", matriz[i][j]);
        }
    }
}

int main() {
    // Inicialización
    int miMatriz[2][1] = {
        {1, 2, 3},
        {4, 5, 6}
    };
    imprimirMatriz(miMatriz);
    return 0;
}
```

Pointers

Los punteros son variables que almacenan la dirección de memoria de otro objeto. Las fuentes destacan los siguientes puntos:

- **Dirección e inicialización:** Se usa & (*adress of*) para obtener la dirección y * para declarar un puntero.
- **Desreferenciación:** Acceder al valor almacenado en la dirección que apunta el puntero.
- **Aritmética de punteros:** Es posible sumar o restar valores a un puntero para navegar por la memoria (común en arrays).
- **Puntero NULL:** Indica que el puntero no apunta a ninguna dirección válida.

```
int main() {  
    int variable = 42;  
    int *ptr = &variable; // Inicialización con la dirección de variable  
  
    printf("Valor: %d\n", *ptr); // Desreferenciación  
  
    int *pNull = NULL; // Puntero nulo  
  
    int arr[1] = {10, 20, 30};  
    int *pArr = arr;  
    printf("Segundo elemento: %d\n", *(pArr + 1)); // Aritmética  
  
    return 0;  
}
```

Punteros y arreglos

Los punteros son muy comdos a la hora de trabajar con matrices. Supongamos que tenemos el siguiente arreglo.

```
int arr[ ] = {10, 20, 30}; // indices: 0,1,2  
int *p;
```

Si quiero inicializar el puntero al primer elemento del arreglo, puedo hacer:

```
p = array[0];  
p = arr; // Cualquiera de las 2 manera esta bien
```

Si quiero acceder a otro elemento del array, puedo ir incrementando el indice

```
int e0 = *p; //accedo al 10  
int e1 = *(p+1); //accedo al 20  
int e3 = *(p+2); //accedo al 30
```

Aritmetica de punteros

```
// Pointer arithmetic.
char * pC_1 = (char *) 0x02UL;
char * pC_2;

pC_2 = pC_1 + 1; // Address: 0x03
pC_2 = pC_1 + 2; // Address: 0x04
pC_2 = pC_1 - 1; // Address: 0x01
```

En la aritmética de punteros, cuando sumas o restas un número entero a un puntero, no se suman bytes individuales, sino **unidades del tipo de dato al que apunta el puntero**.

```
int *pI_1 = (int *) 8UL; // 0x08
int *pI_2, *pI_3, *pI_4; // Cada int ocupa 4 bytes

pI_3 = pI_1 + 1; // Address: 12 Step 4 addresses bytes: 0x0C
pI_4 = pI_1 + 2; // Address: 16 Step 4 * 2 addresses bytes. 0x10
pI_2 = pI_1 - 1; // Address: 4 Step 4 addresses bytes. 0x04
```

Asignar e incrementar **despues** el valor del elemento apuntado por el puntero.

```
*p++ = 6;
*(p++) = 6; // Cualquiera de las 2 manera esta bien
```

Equivalencias

```
*(arr + i) //accedo al elemento del indice i
arr[i]

*(p + i) //accedo al elemento del indice i
pi[i]
```

Strings

```
char * p = "batatinhas";
while(p){
    printf()// Iterate until NULL pointer.
}
```


True vs False

Switch

Logical operators

Decrement and increment operators

Comparing floating point values

Bit operators

sizeof operator

Structures

Structure operators

Unions

Dynamic memory allocation on the Heap

Types of includes

Keyword static

Same ways of defining strings

Functions for manipulating strings and general functions

Functions for manipulating files

Function pointers

C99 -stdint.h - Primitive fixed size types

Preprocessor Macros

Technique for defining more than one statement in a preprocessor Macro.

C for Embedded Systems

Keyword const and volatile

Sizes for 32 bit microController

Two ways of making a Menu with strings - char * - and send it to UART.